ERP PORTAL
Software Requirements Specification (SRS)

Education / School Enterprise Resource Planning Portal

Field	Value
Document status	Draft for Senior Approval
Revision	2.0
Date	26 September 2026
Audience	Internal Senior / Technical Review
Technology	React + TypeScript | Java 21 + Spring Boot 3.x | MySQL 8

Approval objective: establish a traceable, testable implementation baseline covering requirements, RBAC, architecture, workflows, data, API contracts, quantified NFRs, privacy/security, testing and release controls.

This revision deliberately removes client-style filler and replaces generic statements with IDs, constraints, measurable targets and acceptance evidence.

Internal implementation and technical approval baseline

1. Purpose, Scope & Assumptions

1.1 Purpose

This SRS converts the reference product behavior into implementable, testable requirements for an education ERP. It is the baseline for development tickets, API implementation, database design, QA, senior review and release decisions.

1.2 In Scope

Domain	Baseline
Identity/access	Login, logout, password lifecycle, users, roles, permissions, audit
Student lifecycle	Enquiry, admission, student/guardian profile, documents, transfer/exit
Academics	Session, class, section, subject, syllabus, homework, timetable
Attendance	Student/staff marking, correction and summaries
Finance	Fee structures, dues, collection, receipts, discounts, expenses
Operations	Inventory, library, hostel, support/tickets
Administration	Reports, settings, imports/exports, audit

1.3 Out of Scope for Release 1

• Native mobile applications; full statutory general ledger; complex third-party integrations; SaaS multi-tenancy unless separately approved.

• Statutory payroll/tax compliance unless explicitly added to the approved backlog.

1.4 Assumptions

• Initial architecture is a modular monolith with MySQL as system of record.

• One institution/branch is assumed initially; branch-aware fields may be retained for future expansion.

• Backend authorization is authoritative; frontend hiding controls is not a security boundary.

• Critical writes are server-validated and transactional.

2. Document Control, Contents & Glossary

Field	Value
Owner	Engineering / Internship Project Team
Reviewer	Senior Engineer / Technical Lead
Source	Supplied reference videos + agreed technical baseline
Change summary	Requirement IDs; data dictionary; quantified NFRs; acceptance criteria; API contract; privacy/compliance; traceability
Change control	Any approved change must identify affected REQ/NFR/AC IDs and revision

2.1 Table of Contents — names only

1. Purpose, Scope & Assumptions
2. Document Control, Contents & Glossary
3. Reference Analysis & Product Baseline
4. Users, Roles & RBAC
5. Architecture & Technology
6. Functional Requirements
7. Business Workflows & Acceptance Criteria
8. Data Model & Data Dictionary
9. API & Integration Contract
10. Security, Privacy & Compliance
11. Quantified Non-Functional Requirements
12. Testing, Traceability & Definition of Done
13. Deployment, Backup & Operations
14. Implementation Plan, Risks & Approval

2.2 Glossary

Term	Meaning
RBAC	Role-Based Access Control
PII	Personally Identifiable Information
RTO/RPO	Recovery Time Objective / Recovery Point Objective
DTO	Data Transfer Object
Scope	Records a role is allowed to access
Acceptance criteria	Observable conditions required for acceptance

3. Reference Analysis & Product Baseline

3.1 Interpretation

The supplied recordings are treated as UI/workflow references rather than a complete requirements source. Navigation, dashboards, tables, filters, forms and role-oriented actions are translated into the baseline below. Behavior not evidenced by the references is explicitly treated as an implementation assumption.

3.2 Baseline Traceability

Reference area	Baseline capability	IDs
Authentication/shell	Login and authenticated application shell	REQ-AUTH-001..004
Dashboard	Role-aware KPIs and recent activity	REQ-ADM-003
Students/admission	Profile, guardian, admission, documents	REQ-STU-001..008
Academics	Session/class/section/subject/syllabus/timetable/homework	REQ-ACA-001..005
Attendance	Student/staff marking, correction, summaries	REQ-ATT-001..005
Finance	Fees, payments, receipts, discounts, expenses	REQ-FIN-001..009
Operations	Inventory, library, hostel, support	REQ-OPS-001..005
Administration	Users, roles, reports, settings, audit	REQ-ADM-001..008

3.3 Engineering Decisions

• Modular monolith first; microservices are not required for the initial scope.

• REST/JSON APIs; React never accesses MySQL directly.

• Flyway/Liquibase controls schema changes.

• OpenAPI and Postman are linked to requirement IDs.

• Sensitive actions produce auditable events.

4. Users, Roles & RBAC

4.1 Authorization Model

User → Role(s) → Permission(s) → Data Scope → Backend API → Allow/Deny. Authorization is evaluated server-side before a protected operation executes.

4.2 Standard Roles

Role	Scope / responsibility
SUPER_ADMIN	Institution-wide security, users, roles, permissions, all modules
ADMIN	Institution/branch operational administration
MANAGEMENT	Dashboards, reports and approval/oversight
TEACHER	Assigned classes/subjects, attendance, homework
ACCOUNTANT	Fees, payments, receipts and finance reports
HR / STAFF_ADMIN	Staff records and attendance
LIBRARIAN	Catalog, issue/return and fines
RECEPTION / ADMISSION	Enquiry, admission, student records
INVENTORY_MANAGER	Items, suppliers and stock
SUPPORT_AGENT	Assigned support tickets
PARENT / GUARDIAN	Linked child information
STUDENT	Own permitted academic/attendance information

4.3 Permission Convention

MODULE:ACTION, e.g. STUDENT:READ, STUDENT:UPDATE, ATTENDANCE:MARK, FEE:COLLECT, FEE_DISCOUNT:APPROVE, REPORT:EXPORT.

4.4 Core rules

• Deny by default when permission is absent.

• 401 = unauthenticated/invalid authentication; 403 = authenticated but unauthorized.

• Data scope applies to list, detail, update, delete and export operations.

• Role/permission changes are audited and follow the approved token/session invalidation strategy.

4.5 RBAC Matrix & Authorization Requirements

Module	Super Admin	Admin	Mgmt	Teacher	Accountant	Parent/Student
Users/Roles	Full	Manage	View	—	—	—
Students	Full	Full	View	Assigned	View	Own/Linked
Academics	Full	Manage	View	Assigned	View	Own/Linked
Attendance	Full	Manage	View	Mark/Assigned	View	Own/Linked
Fees/Payments	Full	Manage	Approve/View	View	Manage	Own/Linked
Discounts	Full	Approve	Approve	—	Approve*	—
Expenses	Full	Manage	Approve	—	Manage	—
Inventory	Full	Manage	View	—	View	—
Library	Full	Manage	View	View	View	Own
Reports	Full	Export	Export/View	Limited	Export	Limited
Settings	Full	Manage	View	—	Limited	—
Audit	Full	View	View	—	—	—

4.6 Traceable authorization requirements

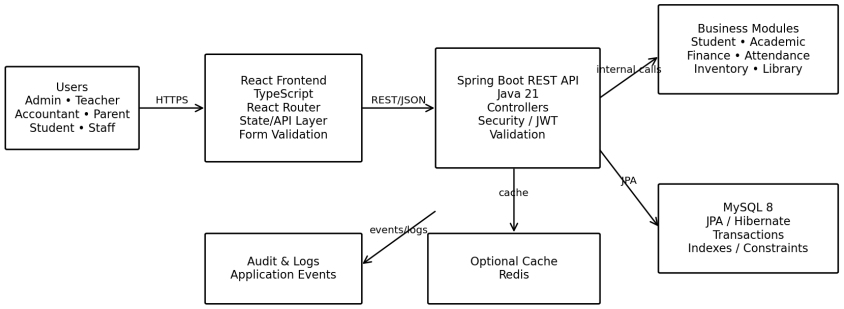
ID	Requirement	Verification
REQ-AUTHZ-001	Protected API operations enforce authentication and permission.	API negative/positive tests
REQ-AUTHZ-002	Record access is restricted by configured data scope.	In-scope vs out-of-scope IDs
REQ-AUTHZ-003	Role/permission administration is restricted and audited.	API + audit test
REQ-AUTHZ-004	Unauthorized requests do not mutate data.	Negative API + DB check
REQ-AUTHZ-005	Exports enforce the same permission and scope rules as reads.	Export test

*Approval rights are configurable; self-approval is prohibited where separation-of-duties is enabled.

5. Architecture & Technology

5.1 Overall Architecture

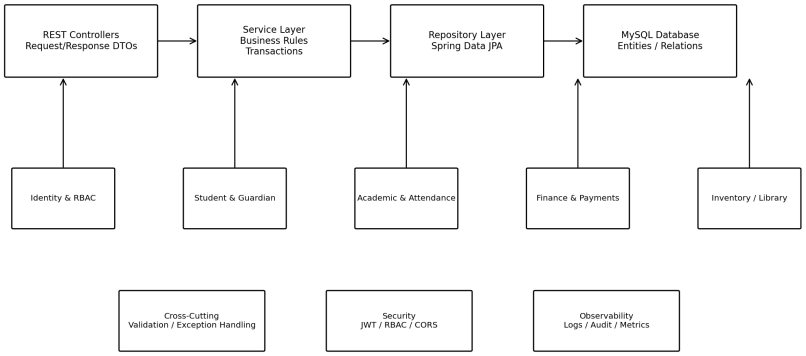
ERP Portal – Overall System Architecture



React/TypeScript provides presentation and client state. Spring Boot owns business rules, validation, authorization and transactions. MySQL is the system of record.

5.2 Backend Architecture

ERP Backend – Modular Monolith / Layered Architecture



Controller → Service → Repository → Database. Security, validation, exceptions, audit, logging and transactions are cross-cutting controls.

5.3 Technology Baseline & Architecture Rules

Layer	Baseline	Rule
Frontend	React + TypeScript + Vite	Route guards; reusable components; client validation
Data fetching	TanStack Query or equivalent	Defined cache/invalidation behavior
Forms	React Hook Form + schema validation	Server validation remains authoritative
Backend	Java 21 + Spring Boot 3.x	Modular monolith
Security	Spring Security	Authentication + endpoint/method authorization
Persistence	Spring Data JPA + Hibernate	DTO/entity separation; service transactions
DB migration	Flyway/Liquibase	Reproducible schema
Database	MySQL 8	FKs, unique constraints, evidence-based indexes
Testing	JUnit 5, Mockito, Spring Boot Test, Testcontainers	Unit/integration/API evidence
API docs	OpenAPI/Swagger	Versioned contract
Deployment	Docker/reverse proxy/TLS	Environment-specific secrets

5.4 Architecture requirements

ID	Requirement
REQ-ARCH-001	Frontend communicates through versioned REST APIs; no direct DB access.
REQ-ARCH-002	Business rules reside in services/domain logic, not only UI/controllers.
REQ-ARCH-003	Schema changes are migration-controlled and reproducible.
REQ-ARCH-004	Critical writes are transactional and deterministic.
REQ-ARCH-005	Common API errors follow the approved error contract.

6. Functional Requirements — Identity & Student

ID	Requirement	Acceptance / evidence
REQ-AUTH-001	Valid user can authenticate and receive authenticated context.	200 + token/session + user context
REQ-AUTH-002	Invalid credentials are rejected without account enumeration.	401 + generic message
REQ-AUTH-003	Protected APIs reject missing/expired authentication.	401
REQ-AUTH-004	Logout follows configured invalidation strategy.	Protected call rejected after logout as designed
REQ-USER-001	Authorized admins create/update/deactivate/view users.	USER:MANAGE tests
REQ-STU-001	Authorized user creates student with mandatory fields.	Missing field → 400
REQ-STU-002	Admission/student identifier is unique.	Duplicate → conflict/validation
REQ-STU-003	Student may have linked guardians under configured rules.	Relationship persists/retrieves
REQ-STU-004	Student lists support search/filter/sort/pagination.	Deterministic page metadata
REQ-STU-005	Student detail is scope-controlled.	Out-of-scope denied/non-disclosed
REQ-STU-006	Admission status follows valid transitions.	Invalid transition rejected
REQ-STU-007	Student documents validate file type/size and metadata.	Invalid upload rejected
REQ-STU-008	Transfer/exit preserves history.	Status changes; history remains

6. Functional Requirements — Academic & Attendance

ID	Requirement	Acceptance / evidence
REQ-ACA-001	Create academic sessions, classes and sections.	Valid create persists
REQ-ACA-002	Assign subjects to classes/sections and teachers.	Assignment visible in teacher scope
REQ-ACA-003	Timetable prevents configured conflicts.	Conflict rejected
REQ-ACA-004	Teachers create homework for assigned scope.	Assigned users can view
REQ-ACA-005	Record syllabus/progress by session/subject.	Progress persists with actor/time
REQ-ATT-001	Assigned teacher/staff can mark attendance.	Only assigned students accepted
REQ-ATT-002	Duplicate attendance is prevented/idempotently handled.	Second submission follows contract
REQ-ATT-003	Attendance correction is restricted and audited.	Old/new value recorded
REQ-ATT-004	Attendance supports date/class/section filters.	Filtered scope correct
REQ-ATT-005	Parent/student sees only permitted history.	Own/linked scope enforced

6.1 Edge cases

• Duplicate marking; locked dates; teacher outside assigned section; timezone/date-boundary behavior; correction after report generation.

• Academic-session changes must not rewrite historical attendance.

6. Functional Requirements — Finance & Operations

ID	Requirement	Acceptance / evidence
REQ-FIN-001	Define fee structures by session/class/category.	Versioned/dated structure
REQ-FIN-002	Generate student dues from applicable structure.	Expected line items/totals
REQ-FIN-003	Authorized cashier posts payment and receipt atomically.	Payment + receipt committed
REQ-FIN-004	Invalid overpayment/amount rules are rejected unless approved path exists.	400/409 as applicable
REQ-FIN-005	Refund/reversal is permission-controlled and audited.	Unauthorized = 403
REQ-FIN-006	Discount/concession follows approval workflow.	Requester cannot self-approve if SoD
REQ-FIN-007	Record expenses with approval state.	Approved expense reports
REQ-FIN-008	Finance exports honor role/date/data scope.	Restricted rows excluded
REQ-FIN-009	Finance totals reconcile to transactions.	Report matches source
REQ-OPS-001	Record inventory items and stock movements.	Balance changes correctly
REQ-OPS-002	Negative stock is rejected unless explicitly configured.	Invalid issue rejected
REQ-OPS-003	Library supports catalog/issue/return/fines.	Availability/history update
REQ-OPS-004	Hostel allocation prevents double allocation.	Conflict rejected
REQ-OPS-005	Support tickets have status, priority, assignee and history.	Lifecycle queryable

6. Functional Requirements — Administration & Reporting

ID	Requirement	Acceptance / evidence
REQ-ADM-001	Authorized admins manage roles and permissions.	RBAC changes affect access
REQ-ADM-002	Role/permission changes create audit events.	Actor/target/action/time
REQ-ADM-003	Dashboards expose role-appropriate KPIs.	No unauthorized KPI/data
REQ-ADM-004	Reports support required filters/date boundaries.	Deterministic results
REQ-ADM-005	Exports are permission-controlled and scope-aware.	Export test
REQ-ADM-006	Bulk imports validate rows and report failures without corruption.	Row-level errors
REQ-ADM-007	Material business settings are audited/versioned where required.	Change attributable
REQ-ADM-008	Audit logs support actor/entity/action/date filters.	Filtered result correct

6.1 Cross-cutting validation

• Mandatory fields return structured field errors; IDs/timestamps are server-generated.

• Sort fields are allow-listed; clients cannot send SQL fragments.

• Historical records use controlled status/soft-delete where retention or referential integrity requires it.

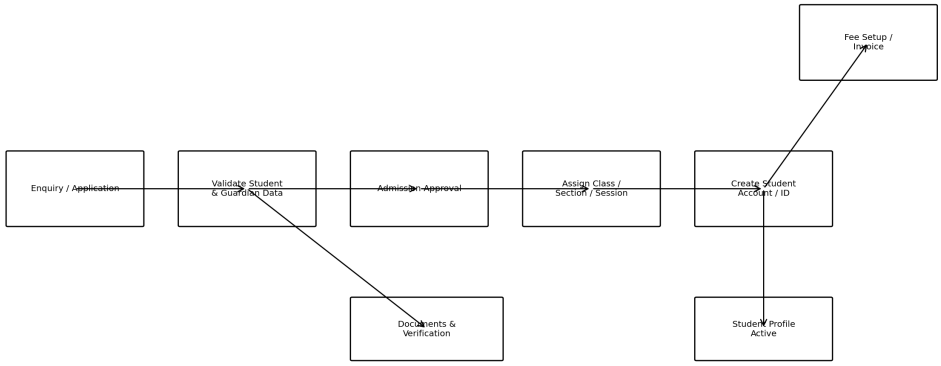
6.2 Definition of Done

A requirement is complete only when implementation, authorization, validation, critical tests, API evidence, reproducible migration, documented errors and acceptance criteria are all demonstrated against its ID.

7. Business Workflows & Acceptance Criteria — Admission

7.1 Student Admission Workflow

Student Admission / Onboarding Workflow



AC ID	Given	When	Then
AC-STU-001	Authorized admission user + new applicant	Submit applicant	Student/admission created with unique ID
AC-STU-002	Existing identifier	Submit admission	Duplicate rejected; no second student
AC-STU-003	User outside student scope	Request detail	Denied/non-disclosed
AC-STU-004	Required document missing	Submit admission	Submission blocked with field error

7.2 Workflow invariants

• Admission state transitions are explicit.

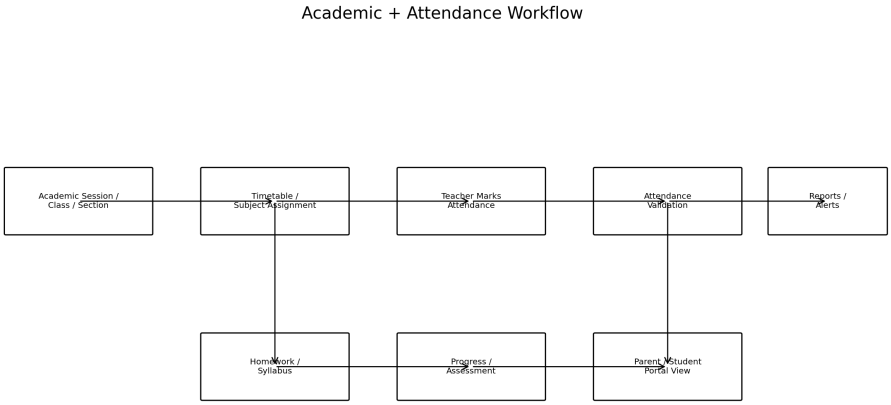
• Student history is retained through transfer/exit.

• Authorization applies at every API step, not only the UI.

7.3 Finance & Attendance Acceptance Criteria

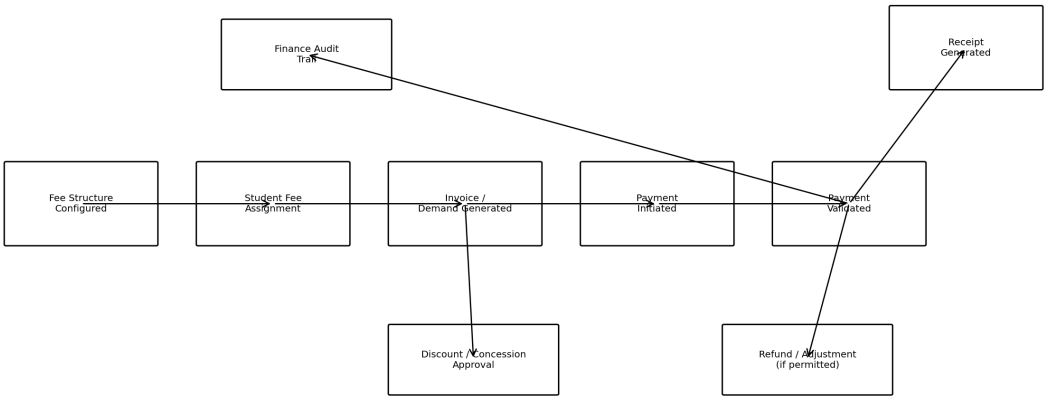
AC ID	Given	When	Then
AC-FIN-001	Valid outstanding fee + cashier	Post payment	Payment and receipt commit atomically
AC-FIN-002	Cashier lacks refund permission	Request refund	403; no financial mutation
AC-FIN-003	Discount requires approval	Submit discount	Pending state; no self-approval where prohibited
AC-FIN-004	Same payment reference submitted twice	Both requests execute	Uniqueness/idempotency prevents duplicate
AC-ATT-001	Teacher assigned to section	Mark attendance	Only assigned students are accepted
AC-ATT-002	Attendance already exists	Submit duplicate	Duplicate prevented/idempotently handled
AC-ATT-003	No correction permission	Request correction	403; original unchanged

7.4 Academic / Attendance Flow



7.5 Finance Flow & Invariants

Fee Collection and Payment Workflow



Financial invariants

• Receipt/payment references are unique.

• Financial writes occur in a transaction.

• Refund/reversal never silently deletes original financial evidence.

• Discounts and expenses record requester, approver, timestamp and outcome.

• Reports derive from transactional records and use the same date/scope rules as APIs.

7.6 Inventory/Library/Hostel invariants

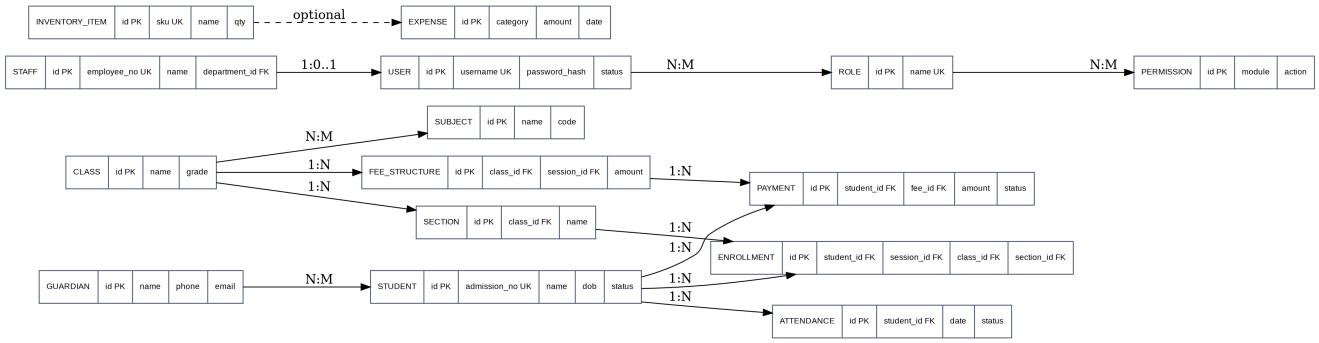
• Stock movement is the source of inventory balance.

• Library issue/return updates availability and borrower history.

• Hostel allocation prevents two active occupants from owning the same bed.

8. Data Model & Data Dictionary

8.1 Conceptual ER Model



The diagram is conceptual. Physical naming may evolve during implementation provided keys, constraints, behavior and API contracts remain equivalent.

8.2 Core entities

Entity	Purpose	Relationships
users	Identity/authentication	roles, audit_logs
roles/permissions	RBAC configuration	users, role_permissions
students	Student master	guardians, admissions, attendance, fees
guardians	Parent/guardian	students
staff	Staff master	attendance, assignments
academic_sessions	Academic period	classes/sections
classes/sections/subjects	Academic structure	students/teachers/timetable
attendance	Presence records	student/staff + session
fee_structures/demands	Charges and dues	student/session
payments/receipts	Financial transactions	student/demand
expenses	Operational expenses	approval
inventory/stock_movements	Stock	items/suppliers
library_items/issues	Circulation	student/staff
audit_logs	Audit evidence	actor + target

8.3 Data Dictionary — Identity & Student

Field	Type	Null	Constraint / meaning
users.id	BIGINT	No	PK, generated
users.username	VARCHAR(100)	No	Unique
users.password_hash	VARCHAR(255)	No	Adaptive hash; never plaintext
users.status	ENUM	No	ACTIVE/LOCKED/DISABLED
roles.code	VARCHAR(50)	No	Unique machine-readable code
permissions.code	VARCHAR(100)	No	Unique MODULE:ACTION
students.id	BIGINT	No	PK
students.admission_no	VARCHAR(50)	No	Unique within institution
students.first_name	VARCHAR(100)	No	Required
students.last_name	VARCHAR(100)	Yes	Optional
students.date_of_birth	DATE	Yes	PII/minor data
students.status	VARCHAR(30)	No	APPLIED/ACTIVE/TRANSFERRED/EXITED
guardians.id	BIGINT	No	PK
guardians.name	VARCHAR(150)	No	Required
guardians.phone	VARCHAR(30)	Yes	PII/contact
guardians.email	VARCHAR(254)	Yes	Validated format
student_guardians.student_id	BIGINT	No	FK
student_guardians.guardian_id	BIGINT	No	FK
student_guardians.relation	VARCHAR(30)	No	Configured relationship

8.4 Data constraints

• Foreign keys protect referential integrity; unique constraints prevent duplicate business identifiers.

• PII fields are minimized and access-controlled.

• Server timestamps are authoritative for audit events; timezone policy must be documented.

8.5 Data Dictionary — Academic, Attendance & Finance

Field	Type	Null	Constraint / meaning
academic_sessions.code	VARCHAR(30)	No	Unique, e.g. 2026-27
classes.id	BIGINT	No	PK
sections.id	BIGINT	No	FK class; unique class+name
subjects.id	BIGINT	No	PK
attendance.id	BIGINT	No	PK
attendance.student_id	BIGINT	No	FK student
attendance.attendance_date	DATE	No	Required
attendance.status	ENUM	No	PRESENT/ABSENT/LATE/etc.
fee_structures.amount	DECIMAL(12,2)	No	>= 0
fee_demands.amount_due	DECIMAL(12,2)	No	>= 0
payments.id	BIGINT	No	PK
payments.student_id	BIGINT	No	FK student
payments.amount	DECIMAL(12,2)	No	> 0 unless adjustment type
payments.reference_no	VARCHAR(80)	No	Unique
payments.status	ENUM	No	POSTED/REFUNDED/REVERSED
expenses.id	BIGINT	No	PK
expenses.amount	DECIMAL(12,2)	No	>= 0
expenses.approval_status	ENUM	No	DRAFT/PENDING/APPROVED/REJECTED
audit_logs.id	BIGINT	No	PK
audit_logs.actor_user_id	BIGINT	No	FK user
audit_logs.action	VARCHAR(100)	No	Business/security action
audit_logs.created_at	TIMESTAMP	No	Server generated

8.6 Indexing policy

Indexes must be based on actual query patterns: unique business identifiers, foreign keys, frequent filter columns and approved report paths. Existence of each index must be verified from migration/schema inspection; Postman cannot prove an index exists.

9. API & Integration Contract

Method	Endpoint	Permission	Purpose
POST	/api/v1/auth/login	Public	Authenticate
POST	/api/v1/auth/logout	AUTH	Logout
GET	/api/v1/me	AUTH	Current user/context
GET	/api/v1/students	STUDENT:READ	Search/filter/page
POST	/api/v1/students	STUDENT:CREATE	Create student
GET	/api/v1/students/{id}	STUDENT:READ	Student detail
PUT	/api/v1/students/{id}	STUDENT:UPDATE	Update student
POST	/api/v1/attendance	ATTENDANCE:MARK	Mark attendance
GET	/api/v1/attendance	ATTENDANCE:READ	Attendance query
GET	/api/v1/fees/demands	FEE:READ	Query dues
POST	/api/v1/payments	FEE:COLLECT	Post payment
POST	/api/v1/payments/{id}/refund	FEE:REFUND	Refund/reverse
GET	/api/v1/reports/{code}	REPORT:READ	Run report
GET	/api/v1/reports/{code}/export	REPORT:EXPORT	Export report
GET	/api/v1/audit	AUDIT:READ	Search audit

9.1 Request/response baseline

POST /api/v1/payments: studentId, demandId, amount, method, referenceNo. Success: 201 + paymentId + receiptNo + status + timestamp. Validation 400; auth 401; authorization 403; missing resource 404; duplicate reference 409.

9.2 Error contract

{"timestamp":"ISO-8601","status":400,"code":"VALIDATION_ERROR","message":"Request validation failed","path":"/api/v1/students","fieldErrors":[{"field":"admissionNo","message":"must be unique"}],"traceId":"..."}

10. Security, Privacy & Compliance

10.1 Security requirements

ID	Requirement	Evidence
REQ-SEC-001	Passwords are adaptively hashed; never plaintext/logged.	DB + code review
REQ-SEC-002	Protected APIs enforce auth + authorization server-side.	API negative tests
REQ-SEC-003	Production traffic uses TLS.	Deployment review
REQ-SEC-004	Input validation and safe error handling prevent leakage.	Security/API tests
REQ-SEC-005	Sensitive actions create audit events.	Audit evidence
REQ-SEC-006	Secrets are environment-managed, not source-controlled.	Config review
REQ-SEC-007	Exports/bulk operations obey scope and permission.	API/export tests

10.2 Education privacy baseline

The portal processes minors' and guardians' personal information. The institution must identify and comply with the privacy/data-protection law applicable to its deployment. For India, the organization should assess the Digital Personal Data Protection Act, 2023 and applicable rules/notifications. FERPA is relevant to U.S. educational institutions and is not assumed to apply to an India-only deployment. This SRS does not provide legal advice.

Control	Baseline
Purpose limitation	Collect only fields required for defined education/operations purposes.
Access	Least privilege; parent/student limited to own/linked records.
Consent/notices	Institution defines lawful basis, notices and parental-consent process where legally required.
Retention	Define periods by student, finance, audit and document category; respect legal holds.
Data requests	Define operational access/correction/deletion process where applicable.
Breach response	Maintain incident escalation and legally required notification process.
Vendor sharing	Approved data-sharing terms and minimum necessary fields.

11. Quantified Non-Functional Requirements

ID	Target	Acceptance method
NFR-PERF-001	p95 API ≤ 500 ms; p99 ≤ 1.5 s under baseline load.	Load test
NFR-PERF-002	List endpoints ≤ 800 ms p95 with indexed filters/pagination.	API/load test
NFR-PERF-003	Exports ≤100,000 rows within 60 s or use async job flow.	Performance test
NFR-CAP-001	5,000 concurrent authenticated sessions; 500 concurrent active API users.	Load test
NFR-CAP-002	100,000 students; 1,000 staff; 5 years attendance/finance baseline dataset.	Representative dataset
NFR-AVAIL-001	Monthly availability ≥99.5%, excluding approved maintenance.	Monitoring/SLA evidence
NFR-REC-001	RPO ≤24 h for standard DB backups.	Backup verification
NFR-REC-002	RTO ≤4 h for restore.	Restore drill
NFR-SEC-001	Critical/high vulnerabilities resolved or explicitly risk-accepted before production.	Security scan
NFR-UX-001	Primary workflows usable at 1280×720 and responsive to 360 px width.	UI test
NFR-OBS-001	Logs include timestamp, level, service, trace ID; secrets/PII excluded.	Log review
NFR-DATA-001	Critical finance/audit records preserve transactional integrity and audit evidence.	DB/integration test

11.1 Capacity note

These are initial engineering targets, not a commercial SLA. Load-test results must be recorded against the actual deployment profile and dataset before production approval. Materially different expected volumes require revised targets and re-approval.

12. Testing, Traceability & Definition of Done

12.1 Test layers

Layer	Purpose	Evidence
Unit	Business rules, validators, mappings	JUnit/Mockito
Integration	JPA, transactions, security, DB	Spring Boot Test/Testcontainers
API/Postman	Auth, RBAC, CRUD, validation, workflow	Collection + results
E2E	Critical UI/API journeys	Automated/manual evidence
Performance	NFR performance/capacity	Load report
Security	Authz, secrets, dependencies, inputs	Scan/test report

12.2 Traceability matrix

Requirement group	Primary evidence	Artifact
REQ-AUTH/AUTHZ	Postman + security integration	Auth/RBAC collection
REQ-STU/ACA/ATT	API + integration + selected E2E	Domain test report
REQ-FIN	Transaction + API + reconciliation	Finance report
REQ-OPS	API + integration	Operations report
REQ-ADM	API + audit/export	Admin report
NFR-PERF/CAP	Load test	Performance report
NFR-REC	Backup/restore drill	Recovery evidence
REQ-SEC	Security tests/scans	Security report

12.3 Postman limits

Postman can prove: HTTP behavior, auth, authorization, CRUD, validation, pagination, filters and observable workflows. **It cannot alone prove:** exact Java class structure, existence of a DB index, internal transaction boundaries, code quality or query plans; use code/schema/migration/DB evidence for those.

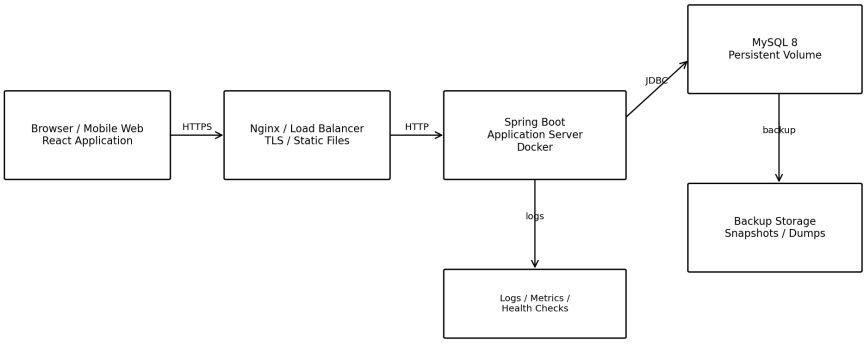
12.4 Definition of Done

Requirement implemented + authorization enforced + critical tests pass + API evidence recorded + migration reproducible + acceptance criteria demonstrated + requirement ID linked to ticket/test.

13. Deployment, Backup & Operations

13.1 Deployment architecture

Production Deployment Architecture



13.2 Environments

Environment	Purpose	Controls
Local	Developer implementation	Non-production data
Test/QA	Integration and acceptance	Seed data + automated tests
Staging	Release candidate	Production-like configuration
Production	Live data	TLS, backups, monitoring, restricted access

13.3 Backup / restore

• Automated MySQL backups must meet RPO ≤24 h and be periodically restored.

• Backups are encrypted and access-controlled; credentials are never stored in source.

• Recovery drills record restore time, data-loss window, validation checks and owner.

13.4 Operations

• Monitor availability, error rate, latency, DB health, disk and backup status.

• Use correlation IDs; logs must avoid passwords, tokens and unnecessary PII.

14. Implementation Plan, Risks & Approval

14.1 Implementation sequence

Phase	Scope	Exit evidence
0	Skeleton, migrations, CI, logging, error contract	Build + migration + CI green
1	Auth, users, roles, permissions, audit	RBAC/API tests
2	Student, guardian, admission, documents	Student acceptance
3	Academics + attendance	Domain tests
4	Fees, payments, discounts, expenses	Reconciliation + authz
5	Inventory, library, hostel, support	Operations acceptance
6	Reports, exports, settings, hardening	NFR/security evidence
7	Staging/UAT/release	Approval checklist

14.2 Risks & mitigations

Risk	Impact	Mitigation
RBAC scope leak	High	Server-side scope tests + senior review
Duplicate finance posting	High	Unique references + transactions + idempotency
PII leakage	High	Logging/export controls + access tests
Schema drift	Medium	Migration tool + CI migration test
Performance degradation	Medium	Evidence-based indexes + load tests
Unclear legal obligations	High	Institution privacy/legal review

14.3 Open decisions before build freeze

• Confirm institution/branch model.

• Confirm JWT/session/refresh-token strategy.

• Confirm external integrations (payments, SMS/email, storage), if any.

• Approve retention and privacy controls for minors, guardians, finance and audit.

• Confirm report formats and approval thresholds for discounts, refunds and expenses.

14.4 Approval Readiness, Sign-off & Master Index

Gate	Evidence	Status
Requirements traceability	Every REQ/NFR/AC has verification path	Ready for review
Architecture	Stack/layers/modules documented	Ready for review
RBAC	Roles, permissions, scope and negative tests	Ready for review
Data	Core entities, fields, types, constraints	Ready for review
API	Endpoint/error contract	Ready for review
NFRs	Performance, capacity, availability, recovery targets	Ready for review
Privacy/compliance	Jurisdiction + retention/consent owner identified	Decision required
Testing	Test layers + traceability + DoD	Ready for review
Operations	Deployment, backup, recovery	Ready for review

14.5 Sign-off

Role	Name	Decision	Date	Signature
Senior Technical Reviewer	_____	Approve / Changes	_____	_____
Project Mentor / Lead	_____	Approve / Changes	_____	_____
Product / Institution Owner	_____	Approve / Changes	_____	_____
Privacy / Compliance Owner	_____	Approve / Changes	_____	_____

14.6 Master requirement index

ID range	Domain	Primary evidence
REQ-AUTH-001..004	Authentication	API + security integration
REQ-AUTHZ-001..005	RBAC	API negative tests + audit
REQ-STU-001..008	Student	API + integration + E2E
REQ-ACA-001..005	Academics	API + integration
REQ-ATT-001..005	Attendance	API + integration
REQ-FIN-001..009	Finance	API + reconciliation
REQ-OPS-001..005	Operations	API + integration
REQ-ADM-001..008	Administration	API + audit/export
REQ-ARCH-001..005	Architecture	Code/config/migration review
REQ-SEC-001..007	Security	Security/config tests
NFR-*	Non-functional	Load/security/recovery/UI evidence

Final approval rule: changes after sign-off must identify affected IDs, rationale, impact on API/data/tests and reviewer. The approved SRS is the baseline; detailed implementation artifacts may refine physical names without changing approved behavior.